

Proyecto Final

Adrián Herrera Morales

Parte 1: Comprensión del Proyecto

Tarea 1.1: Análisis de la arquitectura

Estudia el diagrama de arquitectura proporcionado.

Responde en tu documentación:

¿Cuántos servicios componen el sistema?

El sistema está compuesto por Nginx, PHP, MySQL, Redis y phpMyAdmin.

¿Qué función cumple cada servicio?

Nginx, por un lado, actúa como la puerta de entrada. Recibe las peticiones de los usuarios y las redirige al servicio correspondiente (el blog o phpMyAdmin).

PHP ejecuta la lógica de la aplicación, procesa los datos y genera el contenido HTML.

MySQL almacena de forma persistente los usuarios, posts y comentarios.

Redis almacena temporalmente los resultados de las consultas a la base de datos para que la aplicación sea más rápida (caché).

phpMyAdmin proporciona una interfaz visual para gestionar las tablas de MySQL sin usar la terminal.

¿Cómo se comunican los servicios entre sí?

Se comunican a través de una red virtual interna de Docker.

Nginx -> PHP: Nginx reenvía las peticiones mediante el protocolo HTTP a través del nombre de servicio blog_backend.

PHP -> MySQL: El código PHP se conecta a la base de datos usando el host db.

PHP -> Redis: El código PHP se conecta a Redis usando el host redis.

Nginx -> phpMyAdmin: Nginx redirige las rutas /phpmyadmin/ al contenedor correspondiente.

¿Por qué usar un proxy inverso?

Principalmente porque:

Sirve de punto único de entrada: El usuario sólo necesita acceder al puerto 80 y nginx decide si la petición es para el blog o para el panel de control de la base de datos.

Abstracción: El usuario nunca sabe en qué puerto o IP interna real están corriendo los servicios de PHP o MySQL; solo ve el "localhost".

Seguridad y Control: Permite manejar logs centralizados y, en el futuro, facilitaría añadir certificados SSL (HTTPS) en un solo lugar.

¿Qué ventajas tiene esta arquitectura?

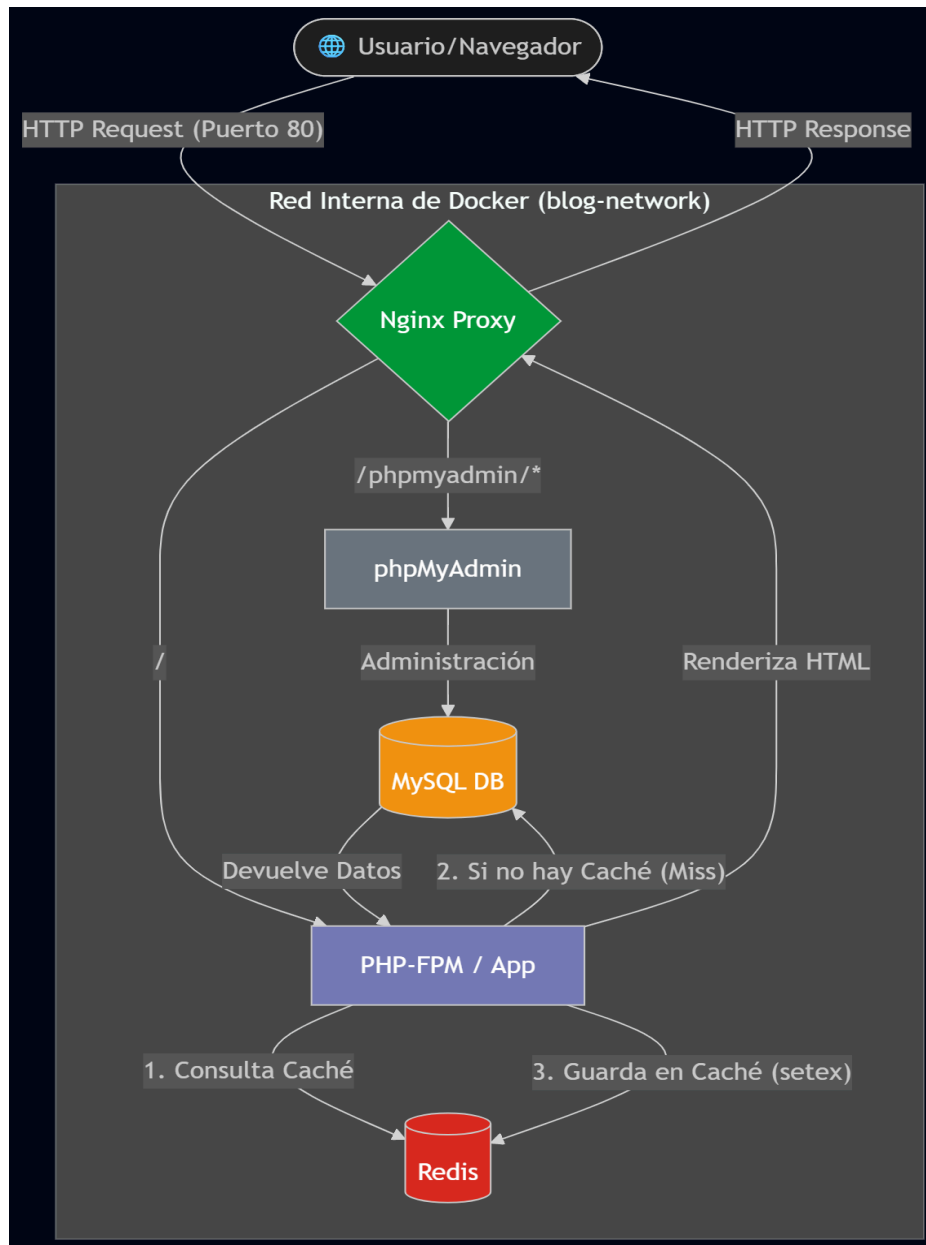
Escalabilidad: Podrías levantar 3 contenedores de PHP si el tráfico aumenta, y Nginx podría repartir la carga entre ellos.

Aislamiento: Si Redis falla, el blog sigue funcionando (el código PHP está diseñado para ignorar el fallo y consultar directamente a MySQL).

Mantenibilidad: Puedes actualizar la versión de MySQL sin afectar la configuración del servidor web Nginx.

Eficiencia: El uso de Redis reduce drásticamente el tiempo de respuesta al no tener que consultar el disco duro para cada visita.

Dibuja un diagrama de flujo



Tarea 1.2: Análisis del código

Lee detenidamente todo el código PHP proporcionado.

Identifica y documenta:

En `index.php`:

¿Cómo se conecta a la base de datos?

Se realiza llamando a la función `getDBConnection()` importada de `database.php`.

¿Cómo funciona el sistema de caché con Redis?

El script empieza generando una clave única.

Primero intenta obtener los datos de Redis con `$redis->get($cacheKey)`.

Si no hay datos, ejecuta la consulta SQL, convierte los resultados a JSON y los guarda en Redis por 300 segundos (5 minutos) usando `setex`.

Si hay datos, los decodifica de JSON a un array de PHP, evitando consultar a MySQL.

¿Qué pasa si Redis no está disponible?

Gracias al bloque `try-catch` en `redis.php`, si la conexión falla, se devuelve un objeto mock (una clase anónima) que simula los métodos de Redis pero siempre devuelve `false`.

¿Cómo se obtienen las estadísticas?

Se realizan consultas directas de conteo (`COUNT(*)`) a las tablas `users` y `comments` mediante el objeto `$db` para mostrarlas en la cabecera del sitio.

En database.php y redis.php:

¿Cómo se leen las variables de entorno?

Se leen utilizando la función `getenv()`. Esto es una buena práctica de microservicios, ya que permite cambiar la configuración sin modificar el código.

¿Qué valores por defecto se usan?

Host: db, Nombre: blogdb, Usuario: bloguser, Password: blogpass.

Host: redis, Puerto: 6379.

¿Cómo se maneja el error de conexión?

En init.sql:

¿Qué relaciones hay entre las tablas?

Un usuario puede tener muchos posts.

Un post puede tener muchos comentarios

Un usuario puede realizar muchos comentarios

¿Qué índices se crean y por qué?

Se crean para acelerar las búsquedas frecuentes. Los que hay son:

`idx_username`: Para hacer búsquedas rápidas durante el login.

`idx_created`: Para ordenar rápidamente los posts y comentarios por fecha de publicación.

`idx_user` / `idx_post`: Para optimizar las consultas que unen (JOIN) las tablas.

¿Cuántos datos de ejemplo se insertan?

Se insertan 4 Usuarios, 8 Posts y 10 Comentarios.

Parte 2: Construcción de Imágenes Docker

Tarea 2.1: Imagen del Frontend

Crear un Dockerfile para el servicio frontend

Código Dockerfile:

```
1  # 1. Usamos una imagen base ligera de PHP 8.2 con FPM (FastCGI Process Manager)
2  FROM php:8.2-fpm-alpine
3
4  # 2. Instalamos las dependencias del sistema y extensiones de PHP
5  # pdo y pdo_mysql son nativas, redis debe descargarse vía PECL
6  RUN apk add --no-cache $PHPIZE_DEPS \
7      && pecl install redis \
8      && docker-php-ext-enable redis \
9      && docker-php-ext-install pdo pdo_mysql
10
11 # 3. Establecemos el directorio de trabajo dentro del contenedor
12 WORKDIR /var/www/html
13
14 # 4. Copiamos el código de la aplicación
15 # Nota: Según tu estructura, copiamos 'public' y 'config'
16 COPY ./public ./
17 COPY ./config ./config
18
19 # 5. Configuramos las variables de entorno por defecto
20 ENV DB_HOST=db \
21     DB_NAME=blogdb \
22     DB_USER=bloguser \
23     DB_PASS=blogpass \
24     REDIS_HOST=redis
25
26 # 6. Exponemos el puerto 9000 (puerto por defecto de PHP-FPM)
27 EXPOSE 9000
```

Tarea 2.2: Construcción de la imagen

Construye la imagen del frontend

```
PS C:\Users\elena\repositorio_git\Repositorios_2526\ProyectoFinalDespliegue\frontend> docker build -t adrihm95/microblog-frontend:v1.0 .
[+] Building 99.5s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 887B                               0.0s
=> [internal] load metadata for docker.io/library/php:8.2-fpm-alpine 2.2s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                     0.0s
=> [1/5] FROM docker.io/library/php:8.2-fpm-alpine@sha256:07c16a3949c2102a1351bb3fab79aa36e97f36301ddd60c275a9d0 6.0s
```

Documenta el tamaño de la imagen.

```
PS C:\Users\elena\repositorio_git\Repositorios_2526\ProyectoFinalDespliegue\frontend> docker images | Select-String "microblog-frontend"
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
adrihm95/microblog-frontend:v1.0    372d38b78cf1    581MB    148MB
```

Parte 3: Configuración con Docker Compose

Tarea 3.1: Diseño del docker-compose.yml

Crea un archivo docker-compose.yml que defina todos los servicios

```
Run All Services
services:
  # Servicio Proxy (Nginx)
  proxy:
    image: nginx:alpine
    container_name: blog_proxy
    ports:
      - "${PROXY_PORT}:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - frontend
      - phpmyadmin
    networks:
      - blog_network

  # Servicio Frontend (imagen de PHP)
  frontend:
    build: ./frontend
    image: adrihm95/microblog-frontend:v1.0
    container_name: blog_app
    environment:
      # El formato es: NOMBRE_PARA_PHP=${NOMBRE_ENV}
      - DB_HOST=${DB_HOST}
      - DB_NAME=${MYSQL_DATABASE}
      - DB_USER=${MYSQL_USER}
      - DB_PASSWORD=${MYSQL_PASSWORD}
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PORT=${REDIS_PORT}
    depends_on:
      - db
      - redis
    networks:
      - blog_network
```

```
# Servicio de Base de Datos (MariaDB)
db:
  image: mariadb:10.11
  container_name: blog_db
  environment:
    - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
    - MYSQL_DATABASE=${MYSQL_DATABASE}
    - MYSQL_USER=${MYSQL_USER}
    - MYSQL_PASSWORD=${MYSQL_PASSWORD}
  volumes:
    - db_data:/var/lib/mysql
    - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
  networks:
    - blog_network

# Servicio de Caché (Redis)
redis:
  image: redis:alpine
  container_name: blog_redis
  volumes:
    - redis_data:/data
  networks:
    - blog_network
```

```
# Servicio de administración de BD
D>Run Service
phpmyadmin:
  image: phpmyadmin:latest
  container_name: blog_pma
  environment:
    - PMA_HOST=${PMA_HOST}
    - PMA_USER=${PMA_USER}
    - PMA_PASSWORD=${PMA_PASSWORD}
  depends_on:
    - db
  networks:
    - blog_network

# Definición de Redes y Volúmenes
networks:
  blog_network:
    driver: bridge

volumes:
  db_data:
  redis_data:
```

Tarea 3.2: Variables de entorno

Crea un archivo .env con todas las configuraciones

```
# Base de Datos
MYSQL_ROOT_PASSWORD=rootpassword_seguro
MYSQL_DATABASE=blogdb
MYSQL_USER=bloguser
MYSQL_PASSWORD=blogpass_seguro

# Configuración del Frontend
DB_HOST=db
DB_NAME=blogdb
DB_USER=bloguser
DB_PASS=blogpass_seguro
REDIS_HOST=redis
REDIS_PORT=6379

# phpMyAdmin
PMA_HOST=db
PMA_USER=bloguser
PMA_PASSWORD=blogpass_seguro

# Puertos
PROXY_PORT=80
PHPMYADMIN_PORT=8081
```

Parte 4: Despliegue y Pruebas

Construye todas las imágenes necesarias y despliega el sistema completo con Docker Compose.

docker-compose up -d --build

Observa los logs de todos los servicios

Logs de la base de datos:

```
PS C:\Users\elena\repositorio_git\Repositorios_2526\ProyectoFinalDespliegue> docker logs blog_db
2026-04-28 08:12:32+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:10.11.16+maria-ubu2204 started.
2026-04-28 08:12:33+00:00 [Warn] [Entrypoint]: /sys/fs/cgroup///memory.pressure not writable, functionality unavailable to MariaDB
2026-04-28 08:12:33+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-04-28 08:12:33+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:10.11.16+maria-ubu2204 started.
2026-04-28 08:12:33+00:00 [Note] [Entrypoint]: MariaDB upgrade not required
2026-04-28 8:12:33 0 [Note] Starting MariaDB 10.11.16-MariaDB-ubu2204 source revision 3218602d3100db9ce7a875511a591cddc173cc16 server_uid uMoC8qC8aqvS+hzHy
DAKQwv0bh0= as process 1
2026-04-28 8:12:33 0 [Note] InnoDB: Compressed tables use zlib 1.2.11
2026-04-28 8:12:33 0 [Note] InnoDB: Number of transaction pools: 1
2026-04-28 8:12:33 0 [Note] InnoDB: Using AVX512 instructions
2026-04-28 8:12:33 0 [Note] mariadbd: O_TMPFILE is not supported on /tmp (disabling future attempts)
2026-04-28 8:12:33 0 [Warning] mariadbd: io_uring_queue_init() failed with EPERM: sysctl kernel.io_uring_disabled has the value 2, or 1 and the user of the
process is not a member of sysctl kernel.io_uring_group. (see man 2 io_uring_setup).
create_uring failed: falling back to libaio
2026-04-28 8:12:33 0 [Note] InnoDB: Using Linux native AIO
2026-04-28 8:12:33 0 [Note] InnoDB: innodb_buffer_pool_size_max=128m, innodb_buffer_pool_size=128m
2026-04-28 8:12:33 0 [Note] InnoDB: Completed initialization of buffer pool
2026-04-28 8:12:33 0 [Note] InnoDB: File system buffers for log disabled (block size=4096 bytes)
2026-04-28 8:12:33 0 [Note] InnoDB: End of log at LSN=44808
2026-04-28 8:12:33 0 [Note] InnoDB: 128 rollback segments are active.
2026-04-28 8:12:33 0 [Note] InnoDB: Setting file './ibtmp1' size to 12.000MiB. Physically writing the file full; Please wait ...
2026-04-28 8:12:33 0 [Note] InnoDB: File './ibtmp1' size is now 12.000MiB.
2026-04-28 8:12:33 0 [Note] InnoDB: log sequence number 44808; transaction id 14
2026-04-28 8:12:33 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buffer_pool
2026-04-28 8:12:33 0 [Note] Plugin 'FEEDBACK' is disabled.
2026-04-28 8:12:33 0 [Note] InnoDB: Buffer pool(s) load completed at 260428 8:12:33
2026-04-28 8:12:33 0 [Warning] You need to use --log-bin to make --expire-logs-days or --binlog-expire-logs-seconds work.
2026-04-28 8:12:33 0 [Note] Server socket created on IP: '0.0.0.0', port: '3306'.
2026-04-28 8:12:33 0 [Note] Server socket created on IP: '::', port: '3306'.
2026-04-28 8:12:33 0 [Note] mariadbd: ready for connections.
Version: '10.11.16-MariaDB-ubu2204' socket: '/run/mysqld/mysqld.sock' port: 3306 mariadb.org binary distribution
PS C:\Users\elena\repositorio_git\Repositorios_2526\ProyectoFinalDespliegue>
```

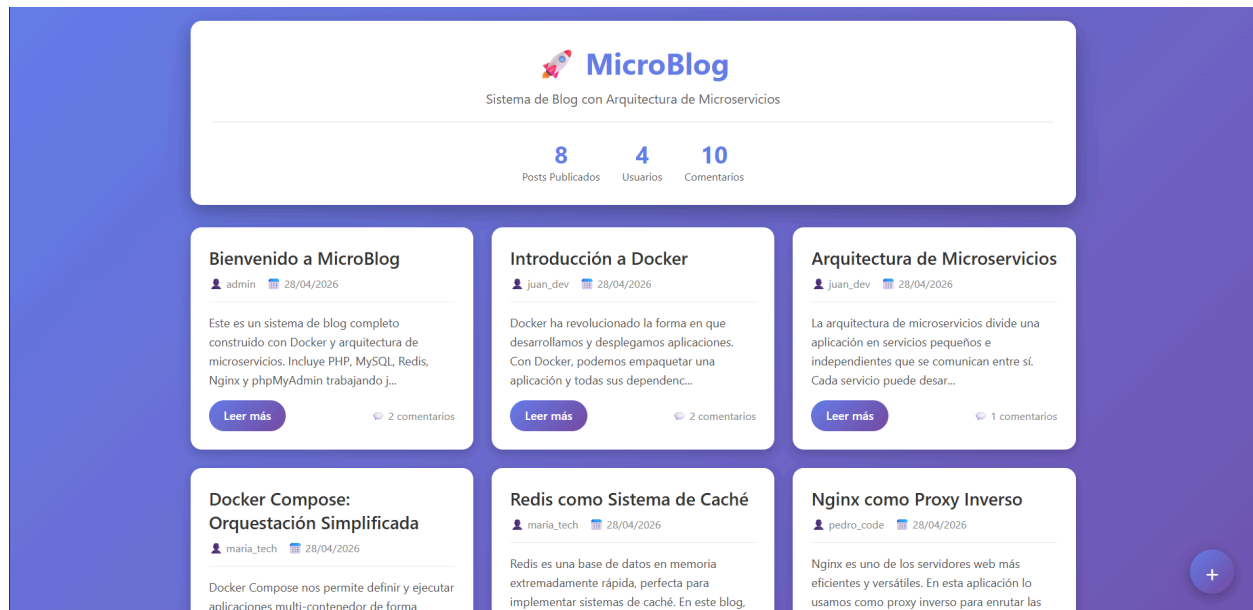
Todo correcto

Logs del proxy:

```
PS C:\Users\elena\repositorio_git\Repositorios_2526\ProyectoFinalDespliegue> docker logs blog_proxy
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
```

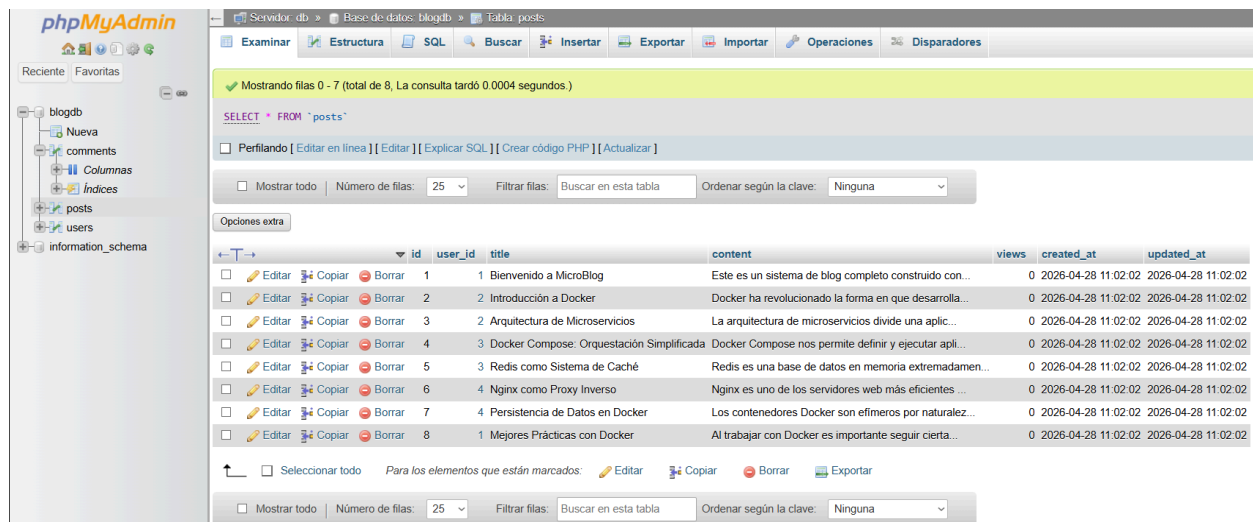
Todo correcto

Acceso al blog



La información y posts son correctos

Acceso a phpMyAdmin



Todas las columnas y filas corresponden con lo esperado

Verificar Caché

Una vez ya he entrado en la web más de una vez, se ve indicado que se están usando datos de la caché

Información del Sistema

Servidor Web
nginx/1.29.8

PHP Version
8.2.30

Base de Datos
10.11.16-MariaDB-ubu2204

Redis
Conectado ✓

Caché activa
Sí (desde caché)

Host

82f0933c53b

Verificación Logs

```
PS C:\WINDOWS\system32> cd C:\Users\elena\repositorio.git\Repositorios_2526\ProyectoFinalDespliegue
PS C:\Users\elena\repositorio.git\Repositorios_2526\ProyectoFinalDespliegue> docker-compose logs --tail=200 proxy frontend db
log_db      | 2026-04-30 14:32:09+00:00 [Note] [Entrypoint]: Entrypoint script for MariaDB Server 1:10.11.16-maria-ubu2204 started.
log_proxy   | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
log_proxy   | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
log_proxy   | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
log_proxy   | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
log_proxy   | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
log_proxy   | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
log_proxy   | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
log_proxy   | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
log_proxy   | /docker-entrypoint.sh: Configuration complete; ready for start up
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:22 +0000] "GET /phpmyadmin/ HTTP/1.1" 200 26024 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:22 +0000] "GET /phpmyadmin/js/messages.php?l-es&v=5.2.3 HTTP/1.1" 200 9025 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:22 +0000] "POST /phpmyadmin/index.php?route=/config/get HTTP/1.1" 200 1432 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:22 +0000] "POST /phpmyadmin/index.php?route=/navigation&ajax_request=1 HTTP/1.1" 200 2020 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:22 +0000] "POST /phpmyadmin/index.php?route=/config/get HTTP/1.1" 200 1523 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:23 +0000] "POST /phpmyadmin/index.php?route=/version-check HTTP/1.1" 200 64 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:23 +0000] "POST /phpmyadmin/index.php?route=/config/set HTTP/1.1" 200 1424 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:24 +0000] "GET /phpmyadmin/themes/pmahomme/img/ajax_clock_small.gif HTTP/1.1" 200 1810 "http://localhost/phpmyadmin/themes/pmahomme/css/theme.css?v=5.2.3" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_app     | [30-Apr-2026 14:32:09] NOTICE: fpm is running, pid 1
log_app     | [30-Apr-2026 14:32:09] NOTICE: ready to handle connections
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:24 +0000] "POST /phpmyadmin/index.php?route=/navigation&ajax_request=1 HTTP/1.1" 200 1880 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:24 +0000] "GET /phpmyadmin/themes/pmahomme/img/b_table_add.png HTTP/1.1" 200 571 "http://localhost/phpmyadmin/themes/pmahomme/css/theme.css?v=5.2.3" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:24 +0000] "GET /phpmyadmin/themes/pmahomme/img/b_props.png HTTP/1.1" 200 573 "http://localhost/phpmyadmin/themes/pmahomme/css/theme.css?v=5.2.3" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_db      | 2026-04-30 14:32:09+00:00 [Warn] [Entrypoint]: /sys/fs/cgroup//memory.pressure not writable, functionality unavailable to MariaDB
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:24 +0000] "GET /phpmyadmin/themes/pmahomme/img/b_minus.png HTTP/1.1" 200 123 "http://localhost/phpmyadmin/themes/pmahomme/css/theme.css?v=5.2.3" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:26 +0000] "GET /phpmyadmin/index.php?route=/server/sql HTTP/1.1" 200 13072 "http://localhost/phpmyadmin/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:26 +0000] "GET /phpmyadmin/js/vendor/jquery/jquery.uitablefilter.js?v=5.2.3 HTTP/1.1" 200 1337 "http://localhost/phpmyadmin/index.php?route=/server/sql" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:26 +0000] "GET /phpmyadmin/js/dist/sql.js?v=5.2.3 HTTP/1.1" 200 9703 "http://localhost/phpmyadmin/index.php?route=/server/sql" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:26 +0000] "GET /phpmyadmin/js/dist/makegrid.js?v=5.2.3 HTTP/1.1" 200 17664 "http://localhost/phpmyadmin/index.php?route=/server/sql" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 OPR/130.0.0.0 (Edition std-2)"
log_proxy   | 172.20.0.1 - - [30/Apr/2026:14:32:27 +0000] "POST /phpmyadmin/index.php?route=/navigation&ajax_request=1 HTTP/1.1" 200 2020 "http://localhost/phpmyadmin/index.php?route=/server/sql" "Mozilla/5.0 (M
```

No se aprecia ningún error, solo algunos warnings sin importancia.

Prueba de persistencia

The screenshot shows the phpMyAdmin interface for a database named 'blogdb'. The 'posts' table is selected, and its structure is displayed. The table has columns: 'id' (int(11)), 'user_id' (int(11)), 'title' (varchar(200)), 'content' (text), 'views' (int(11)), 'created_at' (timestamp), and 'updated_at' (timestamp). The 'id' column has a value of 9, 'user_id' has a value of 'juan_dev - 2', 'title' is empty, 'content' contains 'comentario de prueba', 'views' is 0, 'created_at' is empty, and 'updated_at' is 'current timestamp()'. Below the table structure, a console window shows the SQL query:

```
INSERT INTO `posts` (`id`, `user_id`, `title`, `content`, `views`, `created_at`, `updated_at`) VALUES ('9', '2', 'comentario de prueba', 'Contenido comentario de prueba', '0', current_timestamp(), current_timestamp());
```

 A message at the top of the console indicates '1 fila insertada'.

phpMyAdmin

Reciente Favoritas

blogdb

- Nueva
- comments
- Columnas
- Índices
- posts
- users
- information_schema

Servidor: db » Base de datos: blogdb » Tabla: posts

Examinar Estructura SQL Buscar Insertar Exportar Importar Operaciones Dispar

Columna	Tipo	Función	Nulo	Valor
id	int(11)			9
user_id	int(11)			juan_dev - 2
title	varchar(200)			
content	text			comentario de prueba
views	int(11)			0
created_at	timestamp			current timestamp()
updated_at	timestamp			current timestamp()

1 fila insertada.

```
INSERT INTO `posts` (`id`, `user_id`, `title`, `content`, `views`, `created_at`, `updated_at`) VALUES ('9', '2', 'comentario de prueba', 'Contenido comentario de prueba', '0', current_timestamp(), current_timestamp());
```

[Editar en línea] [Editar] [Crear código PHP]

Vemos como el post se muestra correctamente:

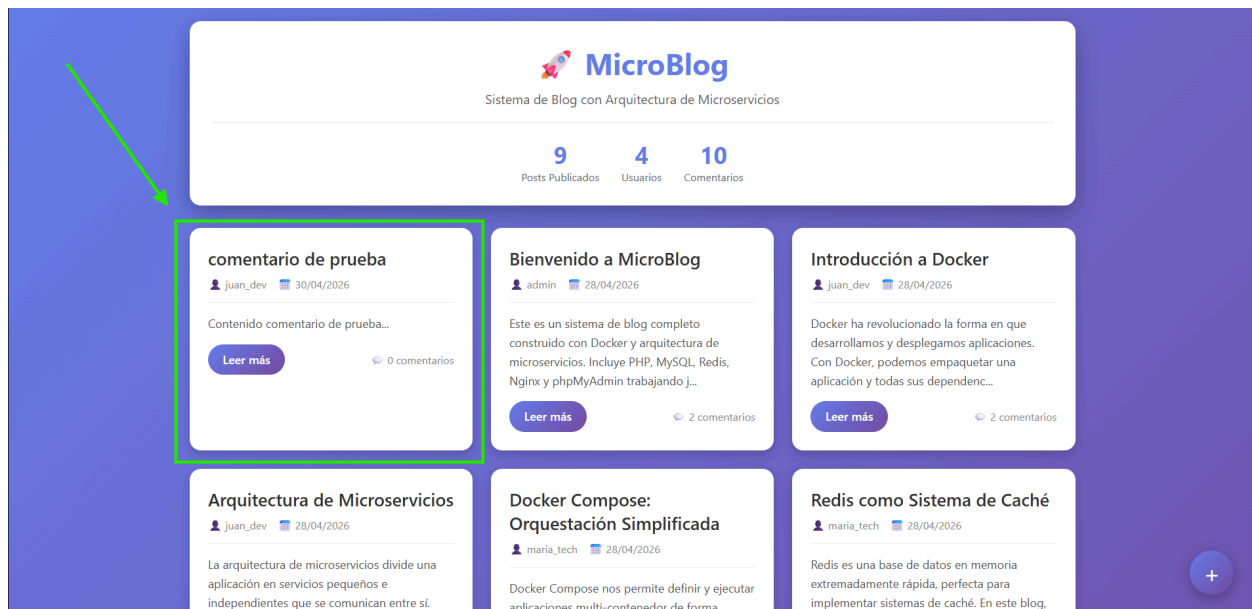


Detengo los contenedores y los vuelvo a levantar:

```
docker-compose down
```

```
docker-compose up -d
```

El contenido sigue estando:



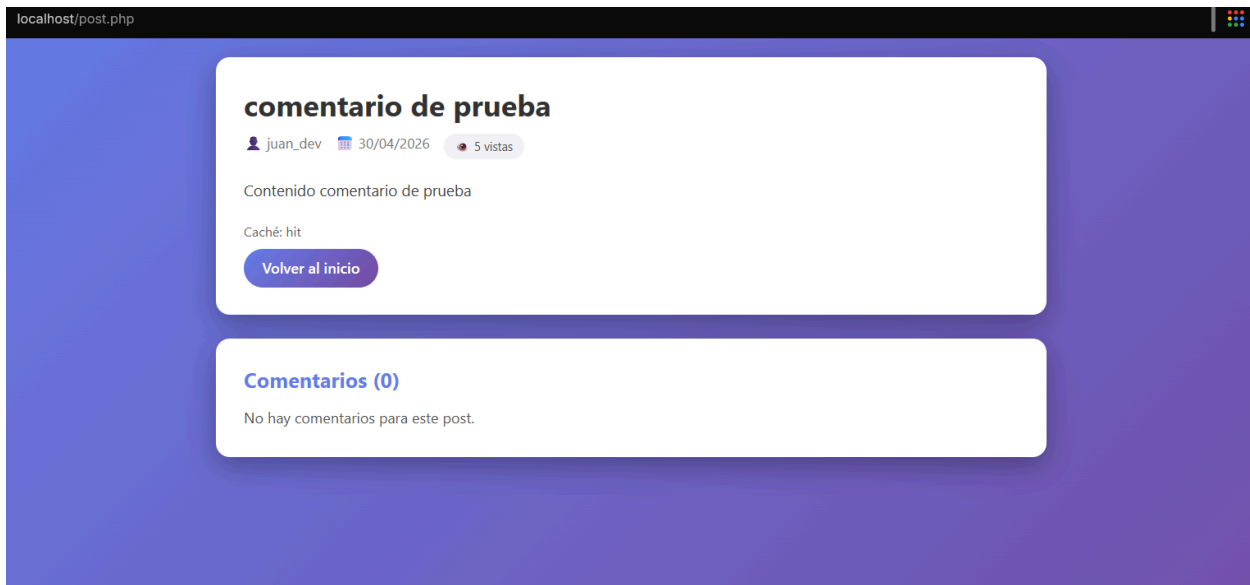
La filas siguen siendo las mismas en la base de datos:

← →			id	user_id	title	content	views	created_at	updated_at
Editar Copiar Borrar	1	1	Bienvenido a MicroBlog	Este es un sistema de blog completo construido con...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	2	2	Introducción a Docker	Docker ha revolucionado la forma en que desarrolla ...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	3	2	Arquitectura de Microservicios	La arquitectura de microservicios divide una aplic...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	4	3	Docker Compose: Orquestación Simplificada	Docker Compose nos permite definir y ejecutar apli...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	5	3	Redis como Sistema de Caché	Redis es una base de datos en memoria extremadamen...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	6	4	Nginx como Proxy Inverso	Nginx es uno de los servidores web más eficientes ...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	7	4	Persistencia de Datos en Docker	Los contenedores Docker son efimeros por naturalez...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	8	1	Mejores Prácticas con Docker	Al trabajar con Docker es importante seguir cierta...	0	2026-04-28 11:02:02	2026-04-28 11:02:02		
Editar Copiar Borrar	9	2	comentario de prueba	Contenido comentario de prueba	0	2026-04-30 14:50:32	2026-04-30 14:50:32		

Parte 5: Funcionalidades Adicionales (Ampliación)

Tarea 5.1: Página de detalle de post

(Por limpieza del documento, he dejado el código fuera del documento, ya que esta visible en el repositorio)



Tarea 5.2: API REST

GET /api/posts - Listar posts

```
Dar formato al texto ☒
{
  "id": 9,
  "user_id": 2,
  "title": "comentario de prueba",
  "content": "Contenido comentario de prueba",
  "views": 6,
  "created_at": "2026-04-30 14:50:32",
  "updated_at": "2026-04-30 16:47:25",
  "username": "juan_dev",
  "comment_count": 0
},
{
  "id": 1,
  "user_id": 1,
  "title": "Bienvenido a Microblog",
  "content": "Este es un sistema de blog completo construido con Docker y arquitectura de microservicios. Incluye PHP, MySQL, Redis, Nginx y phpMyAdmin trabajando juntos en perfecta armonía. Esta es una demostración de cómo los contenedores Docker pueden simplificar el despliegue de aplicaciones complejas.",
  "views": 1,
  "created_at": "2026-04-28 11:02:02",
  "updated_at": "2026-04-30 16:10:18",
  "username": "admin",
  "comment_count": 2
},
{
  "id": 2,
  "user_id": 2,
  "title": "Introducción a Docker",
  "content": "Docker ha revolucionado la forma en que desarrollamos y desplegamos aplicaciones. Con Docker, podemos empaquetar una aplicación y todas sus dependencias en un contenedor que puede ejecutarse en cualquier sistema que tenga Docker instalado. Esto elimina el famoso problema de 'en mi máquina funciona'.",
  "views": 0,
  "created_at": "2026-04-28 11:02:02",
  "updated_at": "2026-04-28 11:02:02",
  "username": "juan_dev",
  "comment_count": 2
},
{
  "id": 3,
  "user_id": 2,
  "title": "Arquitectura de Microservicios",
  "content": "La arquitectura de microservicios divide una aplicación en servicios pequeños e independientes que se comunican entre sí. Cada servicio puede desarrollarse, desplegarse y escalarse de forma independiente. Docker es perfecto para implementar microservicios ya que cada servicio puede vivir en su propio contenedor.",
  "views": 0,
  "created_at": "2026-04-28 11:02:02",
  "updated_at": "2026-04-28 11:02:02",
  "username": "juan_dev",
  "comment_count": 1
},
}
```

GET /api/posts/:id - Obtener post

```
Dar formato al texto ☒
{
  "post": {
    "id": 1,
    "user_id": 1,
    "title": "Bienvenido a Microblog",
    "content": "Este es un sistema de blog completo construido con Docker y arquitectura de microservicios. Incluye PHP, MySQL, Redis, Nginx y phpMyAdmin trabajando juntos en perfecta armonía. Esta es una demostración de cómo los contenedores Docker pueden simplificar el despliegue de aplicaciones complejas.",
    "views": 1,
    "created_at": "2026-04-28 11:02:02",
    "updated_at": "2026-04-30 16:10:18",
    "username": "admin"
  },
  "comments": [
    {
      "id": 1,
      "post_id": 1,
      "user_id": 2,
      "content": "¡Excelente proyecto! Me encanta cómo integra todas estas tecnologías.",
      "created_at": "2026-04-28 11:02:02",
      "username": "juan_dev"
    },
    {
      "id": 2,
      "post_id": 1,
      "user_id": 3,
      "content": "¡Muy útil para aprender Docker. ¿Tienes el código en GitHub?",
      "created_at": "2026-04-28 11:02:02",
      "username": "maria_tech"
    }
  ]
}
```

GET /api/stats - Estadísticas del sistema

```

Dar formato al texto
{
  "posts": 9,
  "users": 4,
  "comments": 10,
  "views": 1,
  "requests": 97,
  "avg_response_ms": 10.4,
  "cache_hits": 55,
  "cache_misses": 15,
  "services": {
    "db": "ok",
    "redis": "ok"
  },
  "timestamp": "2026-04-30T16:55:17+00:00"
}
```

Parte 6: Optimización y Escalado

Tarea 6.1: Optimización de imágenes

Optimiza tu Dockerfile del frontend

Rediseñar el Dockerfile del frontend para dividir el proceso en dos etapas: construcción (donde se instalan dependencias y se compilan las extensiones de php) y la final, en la que simplemente se copian los archivos necesarios.

Implementa .dockerignore apropiado.

```
frontend >  .dockerignore
1  .git
2  .gitignore
3  docker-compose.yml
4  Dockerfile
5  .env
6  node_modules
7  npm-debug.log
8  README.md
9  vendor
10 tests
11 composer.json
12 composer.lock
13
```

Documenta las mejoras obtenidas.

La principal mejora es que hemos reducido mucho el tamaño de las imágenes

Versión	Descripción	Tamaño en Disco (Uso)	Tamaño de Contenido
v1.0	Imagen Original	581 MB	148 MB
v1.1	Imagen Optimizada	122 MB	33.9 MB

Tarea 6.2: Healthchecks

implementar healthchecks en todos los servicios para garantizar que Docker pueda monitorear el estado real de cada componente.

Frontend (PHP-FPM)

Se utiliza `cgi-fcgi` para verificar que el proceso FPM responde en el puerto 9000.

healthcheck:

```
test: ["CMD-SHELL", "cgi-fcgi -bind -connect 127.0.0.1:9000 || exit 1"]
```

Proxy (Nginx)

Se utiliza `wget` contra el endpoint `/health` definido en la configuración de Nginx.

healthcheck:

```
test: ["CMD", "wget", "-qO-", "http://127.0.0.1/health"]
```

Base de Datos (MariaDB)

Se utiliza mysqladmin ping.

healthcheck:

```
test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
```

Redis

Se utiliza redis-cli ping.

healthcheck:

```
test: ["CMD", "redis-cli", "ping"]
```

API

Se utiliza curl contra el endpoint /stats.

healthcheck:

```
test: ["CMD", "curl", "-f", "http://localhost:8000/stats"]
```

Tarea 6.3: Escalado horizontal

Usar comando `docker compose up -d --scale frontend=3`

Balanceo de Carga

Nginx utiliza el nombre de servicio `frontend` para redirigir el tráfico. Gracias al DNS interno de Docker, el nombre `frontend` resuelve a las direcciones IP de las 3 réplicas de forma rotativa (Round Robin).

Verificación

Al acceder al `frontend`, se puede observar en la sección "Información del Sistema" que el Host cambia entre los diferentes IDs de contenedor al refrescar la página.

Todos los contenedores se muestran como `healthy` en el estado de Docker Compose.

Configuración dinámica

Se eliminó el `container_name` fijo del servicio `frontend` en `docker-compose.yml` para permitir que Docker genere nombres únicos para cada réplica.

Parte 7: Seguridad

Tarea 7.1: Mejoras de seguridad

Variables de entorno sensibles

He incluido .env en gitignore para que nunca esté subido a github

Contraseñas seguras

Generar contraseñas complejas y largas para la base de datos y el usuario root de MariaDB.

Usuario no root

Se utiliza el usuario www-data.

Ajustar la propiedad de los archivos (chown) para que el proceso PHP pueda leerlos sin ser administrador del contenedor, por lo que, en caso de una vulnerabilidad en el código PHP, el atacante no tendrá permisos de root sobre el sistema de archivos del contenedor.

Red segmentada

Crear dos redes aisladas:

frontend_net: Red para la comunicación entre el Proxy (Nginx) y el Frontend y API.

backend_net: Red privada para la comunicación entre la aplicación y los servicios de datos (Redis y MariaDB)

Tarea 7.2: Auditoría de seguridad

Usar docker scan tu_usuario/microblog-frontend:v1.0

Vulnerabilidades

Los procesos PHP corrían como root.

El contenedor de Nginx tenía acceso directo a la red de la base de datos.

Correcciones

Se eliminó el Proxy de la red backend_net.

Implementación de USER www-data en los Dockerfiles.